

GARCH on Residuals

Conditional Volatility σ_t — When Spread Volatility Clusters

$$\sigma_t^2 = \omega + \alpha \cdot \varepsilon_{t-1}^2 + \beta_G \cdot \sigma_{t-1}^2$$

CORE IDEA — READ THIS FIRST

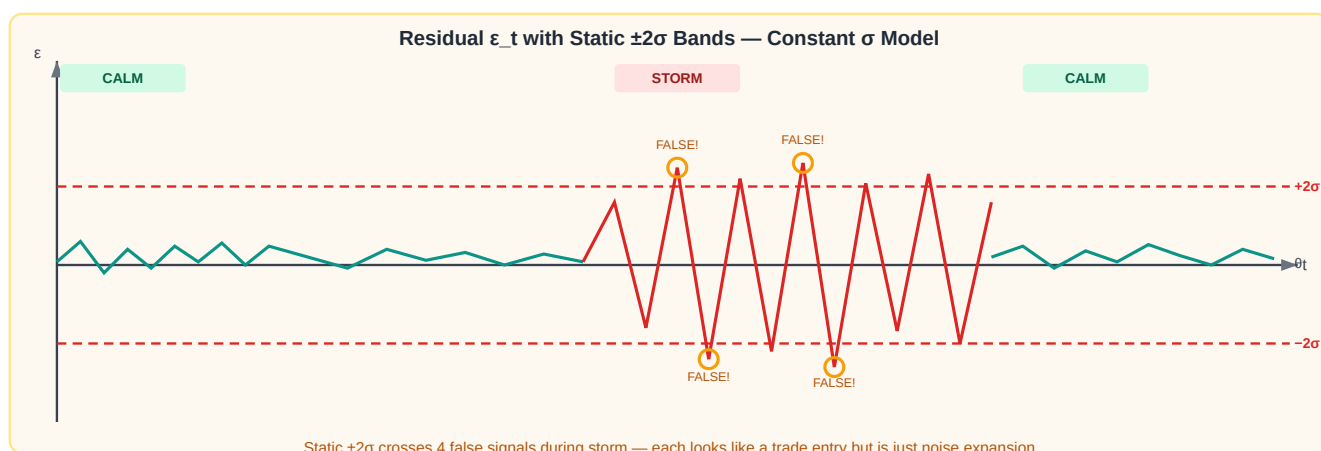
Ornstein-Uhlenbeck assumes **constant** σ . In practice, spread volatility clusters: calm periods are followed by calm, volatile periods by volatile. Using a static z-score during a storm generates false signals. GARCH(1,1) models time-varying conditional variance σ_t , giving a **GARCH z-score** that stays calibrated regardless of market regime.

$$\sigma_t^2 = \omega + \alpha \cdot \varepsilon_{t-1}^2 + \beta_G \cdot \sigma_{t-1}^2 \quad | \quad z_t = (\varepsilon_t - \theta) / \sigma_t$$

7.1 The Problem with Constant σ

Standard OU assumes the residual noise is i.i.d. with fixed σ . In real BTC markets, volatility is highly heteroskedastic — it bunches in time. Static $\pm 2\sigma$ bands drawn at the unconditional standard deviation create two failure modes:

- **Storm false signals:** During high-volatility periods, the spread crosses $\pm 2\sigma$ (static) repeatedly, each crossing appearing to be a trading opportunity. In reality, σ has expanded 3 $\times$, and $2\sigma_{\text{static}}$ is only 0.67 GARCH- σ . These are not true extremes.
- **Calm missed signals:** During quiet periods, a genuine 2σ move is actually a 4σ GARCH event — an extremely rare and high-conviction signal being ignored.



Static bands (red dashes) don't expand during volatile regimes — causing repeated false crossings in the storm period.

7.2 GARCH(1,1) Model

GARCH(1,1) — Generalized AutoRegressive Conditional Heteroskedasticity — updates the variance estimate each period using two inputs: the last shock squared and the last variance estimate.

GARCH(1,1) EQUATION

$$\sigma_t^2 = \omega + \alpha \cdot \varepsilon_{t-1}^2 + \beta_G \cdot \sigma_{t-1}^2$$

ω (omega) — long-run variance floor. Always positive. Prevents σ_t from collapsing to zero.

α (alpha) — shock impact coefficient. Typical range: 0.05–0.15. High $\alpha \rightarrow$ variance reacts quickly to new shocks.

β_G (GARCH beta) — persistence coefficient. Typical range: 0.80–0.90. High $\beta_G \rightarrow$ variance decays slowly (long memory). Note: β_G is distinct from the hedge ratio β .

Stationarity condition: $\alpha + \beta_G < 1$ (otherwise variance explodes to infinity).

Long-run variance: $\sigma^2_\infty = \omega / (1 - \alpha - \beta_G)$

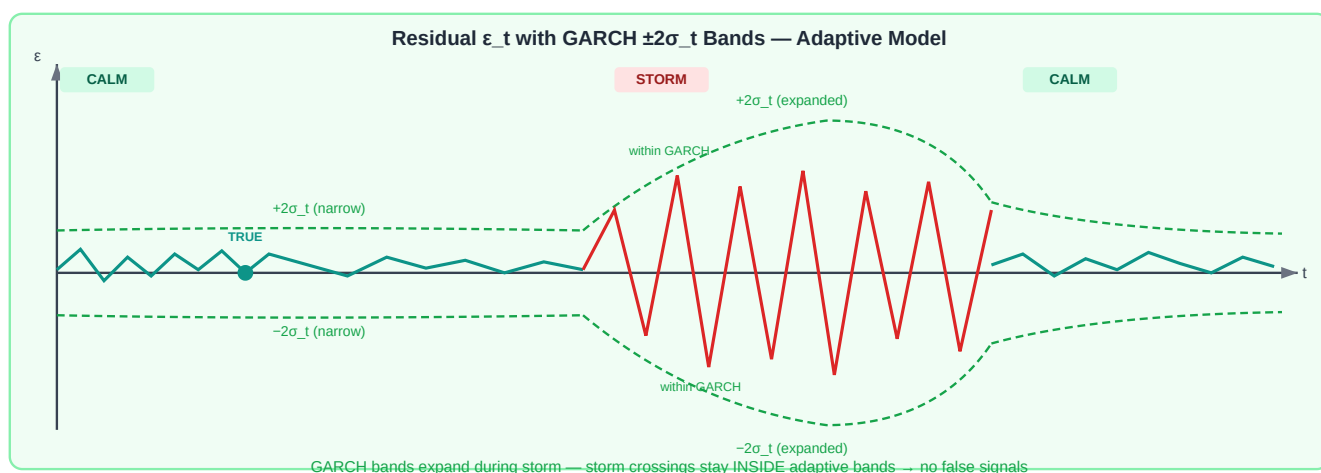
Parameter	Typical Range	Effect of Increasing	Effect of Being Too High
ω (omega)	1e-7 to 1e-4	Raises the variance floor	Overestimates calm-period σ
α (alpha)	0.05 – 0.15	Faster reaction to shocks	Too noisy, σ_t unstable
β_G (GARCH beta)	0.80 – 0.92	Longer variance memory	If $\alpha+\beta_G\geq1$: non-stationary

7.3 GARCH Z-Score — Adaptive Bands

Instead of $z = (\varepsilon_t - \theta) / \sigma_{\text{static}}$, we use the time-varying denominator:

หมายเหตุ: z-score คือสัญญาณหลักของการเข้า/ออก — ตอนนี้อย่าเข้าใจว่า $z = (\varepsilon - \theta) / \sigma$ บอกว่า spread ห่างจากค่ากลางกี่ "ส่วนเบี่ยงเบนมาตรฐาน" ก็พอ — นิยามเต็ม รวมถึง Robust z-score อยู่ใน [บทที่ 10](#)

GARCH z-score: $z_t = (\varepsilon_t - \theta) / \sigma_t$ Entry signal: $z_t > +2.0 \rightarrow$ short spread (expect reversion down) $z_t < -2.0 \rightarrow$ long spread (expect reversion up) Exit signal: $|z_t| < 0.5 \rightarrow$ close position



GARCH adaptive bands (green dashes) expand during the storm — the same residual path produces NO false signals with GARCH z-score.

7.4 When to Use GARCH vs Constant σ

Aspect	Constant σ (OU base)	GARCH(1,1)
Assumption	σ is fixed over time	σ_t varies based on recent shocks
Best for	Very stable pairs, low volatility	Pairs with clustering volatility
Detects regime	No	Yes — σ_t encodes current regime
False signals	High during vol spikes	Low — bands adapt
Computation	Trivial (one number)	O(T) per update, very fast
When to switch	Default	$\text{std}(\text{residuals}, 7\text{d}) / \text{std}(\text{residuals}, 30\text{d}) > 1.5$

Volatility Clustering Trigger

Compute the ratio of 7-day rolling std to 30-day rolling std of the spread residuals. If this ratio exceeds 1.5, the volatility regime has shifted — switch to GARCH z-score immediately. If the ratio drops below 1.1 consistently, constant σ is adequate and GARCH adds minimal benefit.

7.5 Pseudo-Code: GARCH Update & Z-Score

```
function garch_update(omega, alpha, beta_G, prev_sigma_sq, prev_residual): #
  GARCH(1,1) variance recursion – call once per bar # omega: long-run variance floor
  (e.g. 1e-6) # alpha: shock weight (e.g. 0.10) # beta_G: GARCH persistence (e.g. 0.85)
  # prev_sigma_sq:  $\sigma^2_{t-1}$  # prev_residual:  $\varepsilon_{t-1}$  (actual spread residual, not z-
  score) new_sigma_sq = omega + alpha * (prev_residual ** 2) + beta_G * prev_sigma_sq
  return new_sigma_sq function garch_zscore(eps, theta, sigma_t): # eps: current spread
  residual  $\varepsilon_t$  # theta: OU long-run mean  $\theta$  # sigma_t: sqrt(current GARCH variance) z =
  (eps - theta) / sigma_t return z function should_use_garch(residuals,
  window_short=7*24, window_long=30*24): # Returns True if volatility clustering
  detected std_short = rolling_std(residuals, window_short)[-1] std_long =
  rolling_std(residuals, window_long)[-1] ratio = std_short / std_long return ratio >
  1.5 # Main loop (hourly bar) sigma_sq = long_run_variance # initialize to  $\omega/(1-\alpha-\beta_G)$ 
  for each new bar: prev_eps = spread_residual[t-1] sigma_sq = garch_update(omega,
  alpha, beta_G, sigma_sq, prev_eps) sigma_t = sqrt(sigma_sq) z =
  garch_zscore(spread_residual[t], theta, sigma_t) if z > 2.0: execute_short_spread()
  if z < -2.0: execute_long_spread() if abs(z) < 0.5: close_position()
```

7.6 การ Estimate ω , α , β_G จากข้อมูลจริง (MLE)

ค่า ω , α , β_G **ต้องหาจากข้อมูล** ไม่ใช่ guess — วิธีมาตรฐานคือ Maximum Likelihood Estimation (MLE) ผ่าน library ที่ optimize log-likelihood ของ GARCH โดยอัตโนมัติ

```
# Python – ใช้ arch library (pip install arch) from arch import arch_model import
numpy as np # residuals: spread residuals  $\varepsilon_t$  (ใช้ % return หรือ log-diff) # ⚠ UNIT
NOTE: scaling  $\times 100$  เปลี่ยนหน่วย  $\varepsilon$  จาก raw  $\rightarrow$  % ดังนั้น: #  $\omega$  ที่ได้จาก fit จะมีหน่วย %2 (ไม่ใช่
raw2) # ก่อนนำ  $\omega$  ไปใช้ใน garch_update() ต้องหาร 10000 ถ้า code ทำงานกับ raw units # หรือ
scale residuals  $\times 100$  ตลอดทั้ง pipeline ให้สม่ำเสมอ residuals = spread_residuals * 100
# scale เป็น % เพื่อ numerical stability # Fit GARCH(1,1) model =
arch_model(residuals, vol='GARCH', p=1, q=1, mean='Constant') result =
model.fit(disp='off') # disp='off' ปิด iteration output # อ่านพารามิเตอร์ที่ได้ omega =
result.params['omega'] #  $\omega$  (variance floor) alpha = result.params['alpha[1]'] #  $\alpha$ 
(shock weight) beta_G = result.params['beta[1]'] #  $\beta_G$  (GARCH persistence)
print(f" $\omega$ ={omega:.2e},  $\alpha$ ={alpha:.4f},  $\beta_G$ ={beta_G:.4f}") print(f" $\alpha + \beta_G$ =
{alpha+beta_G:.4f} (ต้อง < 1 เพื่อ stationarity)") # Conditional variances ที่ model
fit cond_var = result.conditional_volatility ** 2 # array ของ  $\sigma_t^2$  # Forecast
 $\sigma_{t+1}$  สำหรับ next bar (ใช้ใน live trading) forecast = result.forecast(horizon=1,
reindex=False) sigma_next_sq = forecast.variance.values[-1, 0] sigma_next =
np.sqrt(sigma_next_sq)
```

หน่วย ω : $\times 100$ Scaling ทำให้ ω อยู่ใน %² ไม่ใช่ Raw²

เมื่อ residuals = spread_residuals * 100 (scale จาก fraction เป็น %):

- ε_t มีหน่วย % (เช่น 0.08, 0.15, 0.24)
- ω ที่ fit ได้มีหน่วย %² (เช่น $\omega \approx 0.000006$ %²)
- σ_t จาก GARCH มีหน่วย % $\rightarrow$ GARCH z-score = ε_t (%) / σ_t (%) ✓ ไม่มีปัญหา

ปัญหาถ้า mix หน่วย: ถ้า ε_t ใน code เป็น raw (0.0008) แต่ ω ที่ fit ไว้เป็น %² (0.000006%) $\rightarrow \sigma_t = \sqrt{(\omega\%) \times 0.01} \neq \sigma_t$ ที่ถูก

วิธีแก้ที่ง่ายที่สุด: เลือก scale เดียวตลอด pipeline — แนะนำให้ scale $\times 100$ ทั้งหมด หรือใช้ raw ทั้งหมดโดยหาร $\omega\%$ ด้วย 10000 ก่อนใช้

```
# ถ้าอยากทำงานใน raw units หลังจาก fit ด้วย % units: omega_raw = omega_pct /
10000 # %2  $\rightarrow$  raw2 ( $\div 100^2$ ) # จากนั้น garch_update(omega_raw, alpha, beta, ...)
ด้วย raw epsilon ได้เลย
```

ขนาด Sample ที่จำเป็น

GARCH ต้องการข้อมูลพอ: ≥ 250 observations สำหรับ GARCH(1,1) — น้อยกว่านี้ parameter estimate ไม่เสถียร

ในกรณี crypto perp 1-hour bars: ≥ 10 วัน, 5-minute bars: ≥ 1 วัน

Refit ทุก week หรือเมื่อ residual std เปลี่ยน $> 20\%$ จาก fit ก่อนหน้า

ผลที่คาดหวังสำหรับ BTC Spread

จากประสบการณ์กับ crypto spread โดยทั่วไปพบ:

$\omega \approx 1e-7$ ถึง $1e-5$ | $\alpha \approx 0.05-0.15$ | $\beta_G \approx 0.80-0.92$

ถ้า $\alpha + \beta_G \geq 1 \rightarrow$ model non-stationary $\rightarrow$ ลอง EGARCH หรือ ตัดข้อมูล outlier ออกก่อน refit

ทางเลือก: EGARCH (ถ้า volatility asymmetric)

Crypto มักมี "volatility asymmetry" — ราคาลงทำให้ spread volatile มากกว่าราคาขึ้น

ใช้ EGARCH ได้: `arch_model(residuals, vol='EGARCH', p=1, q=1)`

parameter เพิ่ม γ (asymmetry) — ถ้า $\gamma < 0$ แปลว่า negative shock ทำให้ volatile มากกว่า

Running Example: Bybit BTC Perp ↔ Lighter — Funding Rate War (3 Days)

During a 3-day BTC funding rate divergence event, the Bybit ↔ Lighter spread becomes much more volatile. We compare static vs GARCH z-scores:

Period	Spread σ (actual)	Static σ (baseline)	Static z-score	GARCH σ_t	GARCH z-score
Normal market	0.08%	0.08%	2.0 (signal)	0.08%	2.0 (signal)
Funding war day 1	0.24%	0.08%	6.3 (FALSE!)	0.24%	2.1 (correct)
Funding war day 2	0.22%	0.08%	5.8 (FALSE!)	0.23%	1.9 (no entry)
Recovery	0.10%	0.08%	2.5 (borderline)	0.11%	2.3 (signal)

Parameters used (residuals scaled $\times 100$ to %): $\omega = 0.000192$, $\alpha = 0.12$, $\beta_G = 0.85$ ($\alpha + \beta_G = 0.97 < 1 \checkmark$). Long-run $\sigma = \text{sqrt}(0.000192 / 0.03) \approx 0.080 \rightarrow$ matches normal-market $\sigma \approx 0.08\%$ in the table.

Naive $z = 0.50\% / 0.08\% = 6.3 \rightarrow$ looks like a huge 6σ trade opportunity GARCH
 $z = 0.50\% / 0.24\% = 2.1 \rightarrow$ mild signal — σ_t has already expanded Correct

action: REDUCE size or skip entry (storm period)

Exercise 7.1 — Compute New σ^2

Given: $\omega = 0.0001$, $\alpha = 0.10$, $\beta_G = 0.85$, $\text{prev}_\sigma^2 = 0.0004$, $\text{prev}_\varepsilon = 0.025$

Q: Compute new σ^2 . Is $\alpha + \beta_G < 1$? What is the long-run σ^2 ?

► [Show Answer](#)

Exercise 7.2 — Naive vs GARCH Signal Conflict

The spread residual shows $\varepsilon = 0.48\%$, $\theta = 0.00\%$. Static $\sigma = 0.14\%$. Current GARCH $\sigma_t = 0.26\%$.

Naive $z = 0.48/0.14 = 3.5$. GARCH $z = 0.48/0.26 = 1.8$.

Q: Should you enter a short-spread position? Explain your reasoning.

► [Show Answer](#)

อ้างอิงสำคัญ (References)

- **Engle, R. F. (1982).** Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007. — บทความต้นฉบับที่นิยาม ARCH model; Nobel Prize 2003
- **Bollerslev, T. (1986).** Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327. — ขยาย ARCH เป็น GARCH(p,q) ที่ใช้ใน Ch.7; $\sigma_t^2 = \omega + \sum \alpha_j \cdot \varepsilon_{t-j}^2 + \sum \beta_j \cdot \sigma_{t-j}^2$
- **Nelson, D. B. (1991).** Conditional heteroskedasticity in asset returns: A new approach. *Econometrica*, 59(2), 347–370. — EGARCH ที่กล่าวถึงใน Ch.7; จัดการ asymmetric volatility response